



Dice Bet

How it works

A plain-language walkthrough of the code and the logic behind it — no Go or TypeScript required to follow along.

Backend: Go · PostgreSQL · WebSocket + REST | Frontend: React · TanStack Start

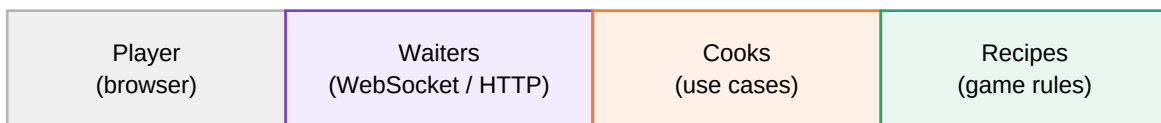
1. What is this thing?

Dice Bet is a small casino-style game: a player bets whether the next roll of a six-sided die will be **even** (2, 4, 6) or **odd** (1, 3, 5). Guess right and the stake doubles; guess wrong and the stake is gone. That's the entire game. What makes the project interesting is not the game itself — it's making sure the money handling around it is bullet-proof: balances never go negative, a bet can't be placed twice by a flaky network connection, and two requests racing each other can't both win.

The project has two halves: a **Go backend** that owns the rules and the money, and a **React frontend** that's just a friendly window into it. This document walks through both, starting with the big picture.

2. The big picture

Think of the backend like a restaurant kitchen. There's an **inner core** that only knows the recipes — it has never met a customer and doesn't know what a phone or a website is. Around that core are the **cooks** who actually carry out an order ("place a bet", "cash out"). And at the very outside are the **waiters** — the part that talks to the customer, whether that's a live chat (WebSocket) or someone filling in a form (HTTP/Postman). The waiters translate what the customer says into something the cooks understand, and translate the result back.



The recipes never point back the other way — the game rules have no idea the pantry is Postgres, or that the waiters speak WebSocket. That's on purpose: it means the rules can be tested completely on their own (no database needed), and either the database or the transport could be swapped out later without touching the rules at all.

*In the code, this is "Clean Architecture": **domain** (the recipes), **service** (the cooks), **transport/ws** and **transport/http** (the waiters), **repository/postgres** (the pantry).*

3. The three things you can do

- 1** **Wallet — "How much do I have?"**
Look up a player's current balance. Read-only, no side effects.
- 2** **Play — "I bet €5 on EVEN"**
Rolls the die right away, decides win or lose, and takes the stake out of the wallet immediately. It does *not* pay out yet — it just tells you the result and how much you'd collect.
- 3** **EndPlay — "Close this round"**
Pays out the pending prize (double the stake on a win, nothing on a loss) and closes the round.

Why split "roll" and "collect" into two separate steps instead of one? It mirrors how a real casino round works — you see the result before you cash out — and it leaves a clean paper trail: every bet has an

obvious "opened" and "closed" moment, which is exactly what makes the safety rules in the next section possible to enforce.

4. The safety rules, in plain English

Money is always whole numbers

Balances and bets are stored as whole cents (500 = €5.00), never as decimals. Computers can't represent amounts like 0.1 or 0.2 exactly in decimal — famously, $0.1 + 0.2$ doesn't equal 0.3 in most programming languages. Counting in whole cents sidesteps that entire class of rounding bugs.

You can't bet money you don't have

Before any bet is accepted, the server checks the stake against the current balance. The database itself also refuses to let a balance go negative, as a second line of defense in case anything upstream got it wrong.

One open bet at a time

You can't start a second bet while the first one is still waiting to be collected. This is enforced twice: once by the application logic, and once by the database itself (it physically refuses to store a second "open" row for the same player).

Repeating a request doesn't repeat the charge

Every bet carries a unique ID generated by the app making the request. If a shaky connection causes the same "place this bet" message to be sent twice, the server recognizes the second one as a repeat — like a tracking number on a parcel: scanning it twice doesn't create two parcels — and simply replies with the original result instead of charging twice.

Two requests at once can't both win the race

If two requests for the same player arrive at almost the same instant (two browser tabs, a retry racing the original), the database briefly locks that player's wallet row while it's being touched — like a fitting room with a single key. The second request simply waits its turn, or is told the first one is already in progress. This even works correctly if the backend is ever running as several separate copies at once, because the lock lives in the database, not in any one server's memory.

5. Walking through a bet, step by step

When you click “Roll”

1

Your browser sends a message

over a live connection (a WebSocket) saying, in effect, “I’m alice, I want to bet 500 cents on EVEN.”

2

The server checks everything first

Does this player exist? Do they already have an open bet? Is the amount within the allowed range and no bigger than their balance?

3

The server rolls the die

using a cryptographically secure random number generator — the same category of randomness used for security keys, not a predictable one — so the result can’t be guessed or rigged.

4

The server settles the bet immediately

decides win or lose, takes the stake out of the wallet right away, and remembers how much is pending to be collected.

5

The server replies

with the roll, the result, and the new balance. All of the above happens inside a single database transaction, so it either all happens or none of it does.

When you click “Collect”

1

Your browser sends a message

saying “close my round.”

2

The server checks there is an open bet

if there isn’t one — e.g. you already collected — it says so rather than doing anything.

3

The server pays out

adds the pending prize to the wallet (zero, if you lost) and marks the round as closed.

6. What can go wrong (and what the app says)

Every rejection has one of a small, fixed set of reasons, sent back as a short code the frontend can react to directly rather than trying to parse a sentence:

Code	Means
INSUFFICIENT_BALANCE	The bet is bigger than the wallet.
INVALID_BET_AMOUNT	Zero, negative, or outside the allowed min/max.
INVALID_BET_TYPE	Anything other than EVEN or ODD.
PLAY_ALREADY_IN_PROGRESS	There's already an open bet for this player.
NO_ACTIVE_PLAY	Tried to collect, but there's nothing open to collect.
CLIENT_NOT_FOUND	That player doesn't exist.
VALIDATION_ERROR	A required field is missing, or a repeated request ID was reused with different bet details.
INTERNAL_ERROR	Something unexpected broke — never leaks database details to the player.

7. The technology, briefly

Piece	What it is	Why
Go	The backend's programming language	Fast, simple, and well-suited to writing reliable network services
PostgreSQL	The database	Battle-tested, and its transactions/locks are exactly what the safety rules above lean on
WebSocket	The game's live connection	Instant back-and-forth without re-opening a connection for every message
REST (HTTP)	A second way to reach the same operations	So the same backend can be exercised from Postman/curl, not only a browser
React + TanStack Start	The frontend	A friendly, responsive interface for playing the game in a browser

8. Where to look in the code

Folder	What lives there
internal/domain	The recipes: game rules, the wallet, error codes. Knows nothing about databases or networks.
internal/service	The cooks: the three operations (Wallet, Play, EndPlay), each running inside one database transaction.
internal/repository/postgres	The pantry: all the actual SQL, talking to PostgreSQL.
internal/transport/ws	One waiter: the live WebSocket connection.
internal/transport/http	The other waiter: the plain REST endpoints, for Postman.
migrations	The database's own blueprint (tables, constraints) as a numbered set of SQL files.
cmd/server	Where everything gets wired together and the server actually starts.
frontend	The React app players actually see and click around in.
test/e2e	Tests that spin up a real database and prove the safety rules hold, even under real concurrency.

9. Running it yourself

The whole thing, backend and database, in one command:

```
docker compose up --build
```

Then, separately, the frontend:

```
cd frontend && npm i && npm run dev
```

That opens the game at **http://localhost:5173**, talking to the backend at **ws://localhost:8080/ws**. A handful of players (alice, bob, carol, dave) are already seeded with starting balances so there's something to play with immediately.

That's the whole story: a small set of rules, enforced twice over (once in the application, once in the database), wrapped in a live connection a browser can talk to. Everything else in the codebase — the tests, the migrations, the Postman collection — exists to keep proving those rules actually hold.